

AI Training Document

Intern Edition — 5-Week Curriculum

Roadmap

We will divide our training process into five steps:

- **WEEK 1**
 - AI Foundations, Python & Math Essentials
 - Python Setup & Core Concepts
 - NumPy, Pandas & Math for ML
- **WEEK 2**
 - Machine Learning Fundamentals
 - Supervised Learning Algorithms
 - Unsupervised Learning
 - Practice Problems
- **WEEK 3**
 - Deep Learning & NLP
 - Neural Networks & CNNs
 - NLP, Transformers & LLMs
 - Practice Problems
- **WEEK 4**
 - Generative AI & Prompt Engineering
 - LLM APIs & Real-world Applications
 - Practice Problems
- **WEEK 5**
 - RAG & Intro to AI Agents
 - Embeddings & Vector Databases

Learning Objectives

After taking this course, you will be familiar with the core concepts of Artificial Intelligence, Machine Learning, Deep Learning, and practical AI application development using modern tools. You will gain hands-on experience building real AI systems and develop the skills needed to contribute to AI projects in a professional environment.

By the end of this course, the trainees should gain the following competencies:

- Understanding of core AI, ML, and deep learning concepts
- Practical experience with Python, NumPy, Pandas, and Scikit-learn
- Ability to build and evaluate machine learning models
- Understanding of NLP fundamentals and how large language models work
- Hands-on experience with prompt engineering and LLM APIs
- Ability to build a basic RAG pipeline for question-answering
- Introduction to AI agent concepts using LangChain

Points to Remember

1. Keep practicing the concepts you learn in all sections on your own. Try to think of examples on your own. Making sure that you understand each concept is your responsibility.
2. For any GitHub repos you create, do not commit in master. Make a branch, add commits to it, and make a PR so code review comments can be tracked. Only merge to master after approval.
3. When you hit an error, read the full error message carefully before asking for help. Most errors tell you exactly what went wrong and where.
4. Note: We will use Python for all training exercises.

WEEK 1: AI FOUNDATIONS, PYTHON & MATH ESSENTIALS

PREREQUISITES & SETUP (WEEK 1 - DAY 1)

Jupyter Notebook

The Jupyter Notebook is an open-source web application that allows you to create and share documents that contain live code, equations, visualizations, and narrative text. It is the standard environment for AI and data science work.

5. What is a Jupyter Notebook?
6. How to use a Jupyter Notebook
7. Setting up your Python environment (pip, virtualenv)

Resources:

- [Jupyter Notebook: An Introduction – Real Python](#)
- [Project Jupyter | Home](#)

Why Python for AI?

Python is the most widely used language for AI and machine learning due to its simple syntax, rich ecosystem of libraries, and strong community support.

8. Features of Python
9. Python 2 vs Python 3
10. Virtual environments and package management (pip, virtualenv)

Resources:

- [Intro to AI with Python](#)

WHAT IS ARTIFICIAL INTELLIGENCE? (WEEK 1 - DAY 2)

Artificial Intelligence (AI) is the simulation of human intelligence processes by computer systems. Machine Learning (ML) is a subset of AI that enables systems to learn from data without being explicitly programmed. Deep Learning (DL) is a further subset of ML that uses multi-layered neural networks.

11. Elements of AI
12. Types of AI: Narrow AI, General AI, Super AI
13. AI use cases: healthcare, finance, NLP, computer vision

Resources:

- [Elements of AI](#)
- [IBM – What is Artificial Intelligence?](#)
- [IBM – Types of AI \(Narrow, General, Super\)](#)
- [Microsoft AI for Beginners](#)

PYTHON FOR AI — CORE CONCEPTS (WEEK 1 - DAY 3)

Python provides the foundation for all AI development. Key concepts include data structures, functions, and object-oriented programming. Focus on understanding these well as they are used throughout every week.

14. Variables, data types, operators
15. Control flow: loops, conditionals
16. Functions and lambda functions
17. Collections: list, tuple, set, dictionary
18. Object-Oriented Programming (OOP): classes and basic inheritance
19. How to read Python error messages and debug your code

Resources:

- [W3Schools Python Tutorial](https://www.w3schools.com/python/)

NumPy, PANDAS & MATH ESSENTIALS (WEEK 1 - DAY 4-5)

NumPy & Pandas

NumPy is the fundamental package for scientific computing in Python. Pandas is a Python library used for working with data sets — it provides functions for analyzing, cleaning, exploring, and manipulating data.

20. NumPy arrays, indexing, slicing, mathematical operations
21. Pandas DataFrames and Series
22. Data loading, cleaning, and transformation
23. Exploratory Data Analysis (EDA)

Resources:

- [NumPy Quickstart – numpy.org](https://numpy.org/doc/stable/user/quickstart.html)
- [Pandas Getting Started – pandas.pydata.org](https://pandas.pydata.org/pandas-docs/stable/10min.html)

Math Essentials for ML

24. Scalars, vectors, matrices — what they are and why ML uses them
25. Matrix operations: addition, multiplication, transpose
26. Gradient descent intuition — how models learn by going downhill
27. Probability distributions: Gaussian, what mean and variance mean
28. Descriptive statistics: mean, standard deviation

Resources:

- [3Blue1Brown – Essence of Linear Algebra](https://www.3blue1brown.com/essence-of-linear-algebra)
- [Linear Algebra](https://www.3blue1brown.com/essence-of-linear-algebra)

Practical Assignment: Implement a simple linear regression model using NumPy and Pandas to predict housing prices based on features like area and number of bedrooms. Preprocess the data, build the model manually using the formula, and evaluate its performance.

WEEK 2: MACHINE LEARNING FUNDAMENTALS

Machine Learning is a branch of Artificial Intelligence that allows machines to learn and improve from experience automatically. It is defined as the field of study that gives computers the capability to learn without being explicitly programmed.

SUPERVISED LEARNING (WEEK 2 - DAY 1-2)

Supervised Learning is a machine learning method where a model is trained on labeled data. The model learns to map input features to output labels. It is classified into two categories: Regression (continuous output) and Classification (discrete categories).

Linear Regression & Logistic Regression

Linear regression predicts continuous numeric values by finding the best-fit line that minimises the difference between predicted and actual values. Logistic regression is a classification algorithm that models the probability of belonging to a class using the sigmoid function.

- 29. Simple and multiple linear regression
- 30. Cost function and gradient descent
- 31. Overfitting, underfitting, and regularisation (L1/L2)
- 32. Sigmoid function and decision boundary
- 33. Multi-class classification: one-vs-rest, softmax

Resources:

- [Google ML Crash Course](#)

Model Evaluation

- 34. Evaluation metrics: accuracy, precision, recall, F1-score, ROC-AUC

Resources:

- [Scikit-learn – Model Evaluation](#)

Decision Trees & Ensemble Methods

Decision trees create a hierarchical structure of decision nodes to make predictions. Random Forest combines multiple decision trees to create a more robust model.

- 35. Decision tree construction (Gini, Entropy)
- 36. Pruning and overfitting mitigation
- 37. Random Forest: bagging and feature randomness
- 38. Feature importance analysis

Note:

XGBoost and Gradient Boosting are powerful extensions of this topic. They are not required for this course but are worth exploring as a bonus once you are comfortable with Random Forest.

Resources:

- [Decision Tree – Scikit-learn Docs](#)

- [Random Forest – Scikit-learn Docs](#)

Support Vector Machines (SVM) & KNN

Support Vector Machines find the optimal decision boundary (hyperplane) between classes. K-Nearest Neighbors predicts the label of a data point based on the majority vote of its K nearest neighbours.

- 39. SVM: hyperplane and margin concept (conceptual overview)
- 40. KNN: distance metrics and choosing K
- 41. Naive Bayes: Bayes theorem for classification

Notebook links:

- [SVM Classifier Tutorial \(Kaggle\)](#)
- [KNN Notebook \(Kaggle\)](#)

UNSUPERVISED LEARNING (WEEK 2 - DAY 3-4)

Unsupervised Learning does not require labeled data. It aims to identify hidden patterns or structures in data.

K-Means Clustering

K-means clustering groups data into clusters based on proximity to centroid points, iteratively updating centroids until convergence.

- 42. K-means algorithm steps and convergence
- 43. Choosing the optimal K: Elbow method and Silhouette score

Resources:

- [K-Means – Scikit-learn Docs](#)

Principal Component Analysis (PCA)

PCA is a dimensionality reduction technique that transforms high-dimensional data into a lower-dimensional representation while preserving the most important information. It is useful for visualisation and feature engineering.

- 44. Variance, covariance matrix, and eigenvectors
- 45. Explained variance ratio
- 46. PCA for visualisation and preprocessing

Notebook link:

- [PCA Tutorial \(Kaggle\)](#)

MODEL EVALUATION & SELECTION (WEEK 2 - DAY 5)

Evaluating machine learning models is critical to understanding their performance and generalisability. Cross-validation helps estimate how a model will perform on unseen data.

- 47. Train/validation/test split

48. K-fold cross-validation
49. Confusion matrix, precision, recall, F1-score
50. Hyperparameter tuning: Grid Search, Random Search

Resources:

- [Scikit-learn Documentation – Model Selection](#)
- [Kaggle ML Courses](#)

Practical Assignment: Build a decision tree classifier using scikit-learn and evaluate its performance on a customer churn prediction dataset. Use Python and scikit-learn to preprocess the data, train the model, and evaluate accuracy, precision, recall, and F1-score. Then apply K-means clustering to a customer segmentation dataset and visualise the clusters using matplotlib.

WEEK 3: DEEP LEARNING & NLP

ARTIFICIAL NEURAL NETWORKS (WEEK 3 - DAY 1)

Artificial Neural Networks (ANNs) are models inspired by biological neurons. Deep learning models learn hierarchical representations of data through multiple layers. This section introduces the fundamental concepts you need to understand how neural networks train.

Neural Network Fundamentals

51. Biological neuron vs artificial neuron
52. Perceptron and multi-layer perceptron (MLP)
53. Activation functions: Sigmoid, Tanh, ReLU, Leaky ReLU, Softmax
54. Forward propagation
55. Backpropagation and the chain rule
56. Loss functions: MSE, Cross-entropy, Binary Cross-entropy

Resources:

- [3Blue1Brown – Neural Networks Series](#)
- [Intro to Deep Learning](#)

Training Techniques

57. Gradient descent variants: Batch, Stochastic, Mini-batch
58. Optimisation algorithm: Adam (primary focus)
59. Learning rate and learning rate scheduling
60. Batch normalisation
61. Dropout and regularisation to prevent overfitting

Resources:

- [Fast.ai Practical Deep Learning for Coders](#)

CONVOLUTIONAL NEURAL NETWORKS & PYTORCH (WEEK 3 - DAY 2-3)

Convolutional Neural Networks are specialised neural networks for processing grid-like data such as images. They use convolutional layers, pooling layers, and activation functions to extract spatial hierarchies of features.

- 62. Convolutional layers: filters, feature maps, strides, padding
- 63. Pooling layers: max pooling, average pooling
- 64. Classic architectures: LeNet, VGG, ResNet
- 65. Transfer Learning: reusing pre-trained models for new tasks

Resources:

- [Andrej Karpathy – Neural Networks: Zero to Hero](#)

PyTorch Basics

PyTorch is the most widely used deep learning framework for research and modern AI development. You will use PyTorch throughout this course.

- 66. Tensors: creation, indexing, operations
- 67. Autograd: automatic differentiation
- 68. Building models with nn.Module
- 69. Training loop: forward pass, loss, backward pass, optimizer step
- 70. Saving and loading models

Resources:

- [PyTorch Official Tutorials](#)

Practical Assignment: Build a CNN model using PyTorch for image classification on the CIFAR-10 dataset. Preprocess the dataset, design the CNN architecture, train the model using backpropagation, and evaluate its performance using accuracy metrics.

NLP FUNDAMENTALS (WEEK 3 - DAY 3-4)

Natural Language Processing (NLP) enables machines to understand and process human language. It involves tasks like tokenisation, text classification, and sentiment analysis.

- 71. Text preprocessing: tokenisation, stop word removal, stemming, lemmatisation
- 72. Part-of-Speech (POS) tagging
- 73. Named Entity Recognition (NER)
- 74. Bag of Words (BoW) and TF-IDF representations
- 75. Sentiment analysis and text classification
- 76. Word embeddings: Word2Vec, GloVe

Resources:

- [Analytics Vidhya – Intro to NLP](#)

RNNs & LSTMs — Conceptual Overview

Recurrent Neural Networks process sequential data by maintaining a hidden state. Long Short-Term Memory (LSTM) networks address the vanishing gradient problem through gating

mechanisms. Understanding these conceptually is important as they are the predecessors to Transformers.

- 77. RNN architecture: hidden state and sequential processing
- 78. The vanishing gradient problem — why it matters
- 79. LSTM: forget gate, input gate, output gate (conceptual)

Resources:

- [Analytics Vidhya – A Brief Overview of RNN](#)

TRANSFORMERS & LARGE LANGUAGE MODELS (WEEK 3 - DAY 5)

The Transformer architecture, introduced in the paper "Attention Is All You Need" (2017), revolutionised NLP by replacing recurrent networks with self-attention mechanisms. BERT uses encoder-only architecture for understanding tasks; GPT uses decoder-only architecture for generation tasks.

- 80. Self-attention and multi-head attention mechanism
- 81. Positional encoding
- 82. Encoder-decoder architecture
- 83. BERT: Masked Language Modelling, bidirectional context
- 84. GPT: Causal language modelling, autoregressive generation
- 85. Tokenisation: BPE, WordPiece, SentencePiece

Resources:

- [3Blue1Brown – Transformers Explained \(YouTube\)](#)
- [Hugging Face LLM Course](#)
- [LLM Course on GitHub – mlabonne](#)

Large Language Models — Overview

Large Language Models like GPT-4, Claude, and Llama are pre-trained on massive corpora and can be adapted to new tasks. For this course, the focus is on understanding how to use them effectively, not on training them from scratch.

- 86. LLM architectures overview: GPT, Llama, Mistral, Gemma
- 87. Pre-training vs fine-tuning vs in-context learning — when to use each
- 88. Using Hugging Face pipeline API for quick text classification and generation

Resources:

- [Hugging Face Transformers Documentation](#)

Practical Assignment: Use the Hugging Face pipeline API to run sentiment classification on a dataset. Compare results before and after loading a task-specific model. Report accuracy and F1-score. No training loop required — focus on using the library correctly.

WEEK 4: GENERATIVE AI & PROMPT ENGINEERING

GENERATIVE AI OVERVIEW (WEEK 4 - DAY 1)

Generative AI refers to systems that can create new content — text, images, code, audio — that resembles human-created content. Modern generative AI is primarily based on large language models and diffusion models.

- 89. Types of generative AI: text, image, code, audio, video
- 90. Diffusion models: DALL-E, Stable Diffusion, Midjourney
- 91. LLM APIs: OpenAI, Anthropic, Google, Cohere
- 92. Capabilities and limitations of LLMs: hallucinations, context windows, knowledge cutoff
- 93. AI safety and responsible AI use

Resources:

- [Cohere LLM University](#)

PROMPT ENGINEERING (WEEK 4 - DAY 2-3)

Prompt engineering is the practice of designing and optimising inputs to language models to achieve desired outputs. It is one of the most immediately practical skills for working with LLMs and is heavily used in real AI products.

- 94. Zero-shot prompting: direct instruction without examples
- 95. Few-shot prompting: providing input-output examples
- 96. Chain-of-Thought (CoT) prompting: step-by-step reasoning
- 97. System prompts, context design, and personas
- 98. Prompt templates and output parsers
- 99. Evaluating and iterating on prompts

Resources:

- [Anthropic Prompt Engineering Guide \(official docs\)](#)
- [Prompt engineering](#)

WORKING WITH LLM APIS (WEEK 4 - DAY 4-5)

Modern AI applications are built by calling LLM APIs. Understanding how to structure API calls, manage context, and get structured outputs are core practical skills for any AI engineer.

- 100. OpenAI API: chat completions and system prompts
- 101. Anthropic API: messages API and system prompts
- 102. Maintaining conversation history across turns
- 103. Structured outputs and JSON mode
- 104. Function calling: letting the model trigger actions

Resources:

- [OpenAI API Documentation](#)
- [Anthropic Claude API Documentation](#)

Practical Assignment: Build a command-line chatbot using the OpenAI or Anthropic API. Implement a clear system prompt, maintain conversation history across multiple turns, and return structured JSON responses for at least one command type.

WEEK 5: RAG & INTRO TO AI AGENTS

RETRIEVAL-AUGMENTED GENERATION — RAG (WEEK 5 - DAY 1-2)

Retrieval-Augmented Generation (RAG) improves LLM responses by grounding them in relevant, often private or up-to-date data retrieved from external sources. Plain LLMs hallucinate, have knowledge cutoffs, and cannot access your organisation's private data. RAG solves all three problems and is the most widely used architecture for enterprise AI applications today.

105. RAG architecture: retriever + knowledge base + LLM generator
106. RAG pipeline: indexing, retrieval, augmentation, generation
107. Why RAG: reducing hallucinations, accessing private and current data
108. Chunking strategies: fixed-size and recursive chunking
109. Document loaders and preprocessing

Resources:

- [freeCodeCamp – RAG Fundamentals & Advanced Techniques](#)
- [freeCodeCamp – RAG from Scratch](#)

EMBEDDINGS & VECTOR DATABASES (WEEK 5 - DAY 3)

Embeddings are numerical vector representations of text that capture semantic meaning. Similar texts have similar embeddings in vector space. Vector databases store and search these vectors efficiently.

110. What are embeddings and why semantic similarity matters
111. Embedding models: OpenAI text-embedding, Sentence-Transformers
112. Vector database: FAISS or Chroma (one to focus on)
113. Similarity search: cosine similarity, dot product, Euclidean distance

Resources:

- [Weaviate Vector Database Learning Center](#)
- [Pinecone Learn – Vector Database Basics](#)

AI AGENTS INTRODUCTION (WEEK 5 - DAY 4-5)

An AI agent is a system that perceives its environment, reasons about it, and takes actions to achieve goals. Unlike simple LLM calls, agents can plan multi-step tasks, use external tools, and maintain memory. This section introduces the core concepts and gives you hands-on experience with LangChain.

114. What is an AI agent: perception, reasoning, action loop

- 115. ReAct architecture: Reason + Act — the most common agent pattern
- 116. Tool use and function calling in agents
- 117. Short-term memory (conversation context) vs long-term memory (vector stores)
- 118. LangChain fundamentals: chains, prompts, tools, simple agents
- 119. Intro to LangGraph: nodes, edges, and stateful workflows (one simple example)

Resources:

- [Hugging Face Agents Course](#)
- [Intro to LangChain](#)
- [Intro to LangGraph](#)

BASIC DEPLOYMENT (WEEK 5 - DAY 5)

Once you have built a working AI application, it needs to be accessible. This section covers the basics of wrapping your model in a simple web API using FastAPI.

- 120. Building a REST API endpoint with FastAPI
- 121. Accepting input and returning predictions via HTTP
- 122. Testing your API locally

Resources:

- [FastAPI Official Documentation](#)

— End of AI Training Document (Intern Edition) —